DAY 13 | HANDS-ON ACTIVITIES

# Testing and Data Engineering Intro: Problem Statements

Seven problems: three on testing, four on data pipelines. Run PyTest with !pytest -q in Colab or pytest -q in the VS Code terminal.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

**How to work:** read the problem, write the steps in plain English first, then write the code. Test with the sample run, then try your own inputs, including wrong ones.

## Problem 1: First Tests

**Level:** Warm-up | **Concepts:** assert, pytest.raises

Write a module with is_valid_email(s), discount(price, pct) and grade(marks). Write at least two tests per function, including one error case.

### Sample run / expected result

```
pytest -q
7 passed in 0.03s
```

### Hints

- Name files test_*.py and functions test_*.

## Problem 2: Parametrize

**Level:** Easy | **Concepts:** pytest.mark.parametrize

Test grade(marks) with one parametrized test covering 0, 49, 50, 69, 70, 84, 85 and 100, plus a test that 101 raises ValueError.

### Sample run / expected result

```
9 passed
```

### Hints

- Boundary values are where bugs hide.

## Problem 3: API Test Suite

**Level:** Medium | **Concepts:** fixtures, TestClient

Write a client fixture and 8 tests for a Tasks API: list, create (201), invalid create (422), get, missing (404), patch, delete (204), delete twice (404).

### Sample run / expected result

```
8 passed
```

### Hints

- Reset the in-memory store in a fixture so tests do not affect each other.

## Problem 4: JavaScript Unit Tests

**Level:** Medium | **Concepts:** node:test, assert

Write formatPrice(n) and cartTotal(items) in JavaScript and test them with Node's built-in test runner.

### Sample run / expected result

```
node --test
# pass 4
# fail 0
```

### Hints

- import test from "node:test"; import assert from "node:assert/strict";

## Problem 5: Messy Data ETL

**Level:** Medium | **Concepts:** pandas extract, transform, load

Given orders.csv with duplicates, blank amounts, mixed date formats and extra spaces, write extract, transform and load functions that produce a clean orders table in SQLite.

### Sample run / expected result

```
Raw rows: 12 -> Clean rows: 8
Loaded 8 rows into shop.db.orders
```

### Hints

- drop_duplicates, str.strip, pd.to_numeric(errors="coerce"), pd.to_datetime(format="mixed")

## Problem 6: ELT Version

**Level:** Challenging | **Concepts:** load raw, transform with SQL

Load the raw CSV as-is into a raw_orders table, then create a clean orders_clean table with a SQL query inside SQLite. Compare row counts with your ETL result.

### Sample run / expected result

```
raw_orders: 12 rows
orders_clean: 8 rows (matches ETL)
```

### Hints

- CREATE TABLE orders_clean AS SELECT DISTINCT ... WHERE amount IS NOT NULL

# Problem 7: Idempotent Incremental Load

**Level:** Stretch | **Concepts:** incremental ingestion, data quality tests

Write load_incremental(new_df, conn) that inserts only orders not already present (by order_id). Prove that running it twice keeps the same row count, and add PyTest data quality tests.

## Sample run / expected result

```
Run 1: inserted 3
Run 2: inserted 0
4 passed
```

## Hints

- Read existing ids with pd.read_sql, then filter with ~df.order_id.isin(ids).

*Ganbatte (頑張って): do your best*